Machine Learning for Physicists

Caio Lagana

August 4, 2026

Contents

Preface

So, you're a physicist looking to dive into the world of machine learning. You have a deep understanding of calculus, are familiar with experimental data-fitting methods, and have mastered concepts like entropy and non-linearity—yet you have only a vague idea of how a machine learning algorithm is actually written and how it works? Then this book is for you. And the good news is: your learning curve will grow fast. I've been in your shoes. I used to view machine learning as a distant horizon, but I soon realized that the knowledge I'd gained in physics made it incredibly easy to quickly master the concepts and engineering behind the field. So, come along and be amazed.

Chapter 1

Few-neuron networks

Toy-models are useless for real-world applications, but are wonderful for didactic purposes. In this chapter, I use few-neuron models to present *the* fundamental concepts for any advanced topic in ML theory.

1.1 Single-neuron ML

You have been using a single-neuron network since the first time you fitted experimental data, but perhaps no one ever told you that. Let's recall the basics. You have a set of N experimental data points

$$(x_i, y_i) \tag{1.1}$$

through which to fit a linear function

$$f(x) = ax + b. \tag{1.2}$$

The Mean Squared Error Loss, defined as

$$\begin{aligned} L_{\mathrm{MSE}}(a, b) &= \frac{1}{N} \sum_{i=1}^{N} [y_i - f(x_i)]^2 \\ &= \frac{1}{N} \sum_{i=1}^{N} [y_i - (ax_i + b)]^2 \end{aligned} \tag{1.3}$$

tells us how distant the curve $f(x)$ is from the datapoints. Fitting $f(x)$ to the datapoints (x_i, y_i) means minimizing $L_{\mathrm{MSE}}(a, b)$ with respect to a and b, that is,

$$\frac{\partial}{\partial(a, b)} L_{\mathrm{MSE}}(a, b) = 0. \tag{1.4}$$

No surprises so far, right? Well, what if I tell you the above is the simplest possible neural network? Indeed, it is a single-neuron network, where (1.2) is the activation function, (1.3) is the loss function and (1.4) is a very rudimentary backpropagation method.

Solving (1.4) for a and b, which, in this simple case is possible analytically, fixes the parameters to a_0 and b_0

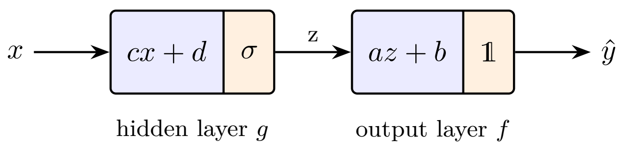
$$\begin{aligned} a_0 &= \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} \\ b_0 &= \bar{y} - a_0 \bar{x} \end{aligned} \tag{1.5}$$

and we end up with a model $f_0(x) = a_0 x + b_0$ (in (1.5), $\bar{x}$ and $\bar{y}$ are the mean values of the x_i and y_i datapoints). This model is a single neuron, no hidden layers: it receives one input parameter and outputs a number. f_0 has learned from data (1.1) through (1.4) and can now predict on any given x. Pretty cool, huh? The concepts were sitting there all the time. Take a moment to re-read the definitions and let's jump to the second simplest model: a two-neuron network.

1.2 Two-neuron ML

Our neuron from Sec. 1.1 was of the form $f(x) = ax + b$ (always think of it as receiving one value x and, upon training for a, b parameters, outputting another value $a_0x + b_0$). Let us now place a second neuron, g, before it, so that g receives the input and f produces the final output. This new neuron will be of the form $g(x) = \sigma(cx + d)$, where $\sigma(x) = 1/(1 + e^{-x})$. The need for σ will be elucidated later; for now, just accept it as a magical ingredient. In fact, it is the so called neuron's activation function!

Placing one neuron before another means composing the two functions, that is, given input x, the network outputs $f(g(x))$. In other words, input x feeds a (hidden) layer g, which applies its weight c and bias d, activates through σ and feeds output layer f, which in turn applies weight a and bias b and outputs $\hat{y}$. Layer f has identity operator $\mathbb{1}$ as its activation function. A visual diagram may help.



Our task now is to optimize the network for parameters a, b, c, d over training data. The loss function is similar to (1.3), just generalized for the two-neuron case,

$$L_{\text{MSE}}(a, b, c, d) = \frac{1}{N}\sum_{i=1}^{N}[y_i - f(g(x_i))]^2. \tag{1.6}$$

This time, however, MSE minimization, stated as

$$\frac{\partial}{\partial(a, b, c, d)} L_{\text{MSE}}(a, b, c, d) = 0 \tag{1.7}$$

cannot be solved analytically as in (1.5), and numerical methods must be used. The recipe is as follows: start at a certain initial position (a, b, c, d) on the four-dimensional parameter space, then slightly change each coordinate by an amount according to[1]:

$$\begin{aligned}
a &\to a - \alpha\frac{\partial L_{\text{MSE}}}{\partial a}\\
b &\to b - \alpha\frac{\partial L_{\text{MSE}}}{\partial b}\\
c &\to c - \alpha\frac{\partial L_{\text{MSE}}}{\partial c}\\
d &\to d - \alpha\frac{\partial L_{\text{MSE}}}{\partial d}
\end{aligned} \tag{1.8}$$

The term α is called the learning rate (haleluiya, a name you've heard before, now in a clear context!) the derivatives of L_{MSE} in (1.8) points to the max uphill on the hypersurface, so the minus sign will move the initial coordinate to a new position where L_{MSE} is lower. After repeating this procedure a couple of times, (1.7) will be solved to a certain desired precision (yes, I am oversimplifying, this step is a billion-dollar industry).

[1] Picture this process as finding a minumum in a graph. For reference, in Sec. 1.1 the graph of the loss function was a two-dimensional paraboloid with hypersurface $(a, b, L_{\text{MSE}}(a, b))$ embedded in $\mathbb{R}^3$; in this two-neuron scenario, the graph is a four-dimensional hypersurface embedded in $\mathbb{R}^5$. The paraboloid has a clear, well-defined minimum; this time, there may be several local minima plus saddle points and plateaus, and which one you land in depends on the starting point.

Let's dive a little into the algebra of this example, as it will allow us to explore one very fundamental concept: backpropagation. The partial derivatives for this case can be taken explicitly:

$$\begin{aligned}
\frac{\partial L}{\partial a} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a} = -\frac{2}{N} \sum_i \left[y_i - f(g(x_i))\right] g(x_i) \\
\frac{\partial L}{\partial b} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial b} = -\frac{2}{N} \sum_i \left[y_i - f(g(x_i))\right] \\
\frac{\partial L}{\partial c} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial c} = -\frac{2a}{N} \sum_i \left[y_i - f(g(x_i))\right] \sigma'(cx_i + d) x_i \\
\frac{\partial L}{\partial d} &= \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial d} = -\frac{2a}{N} \sum_i \left[y_i - f(g(x_i))\right] \sigma'(cx_i + d)
\end{aligned} \tag{1.9}$$

None of these derivatives are useful right now. I do not intent to make predictions on any fake dataset with this toy-model. Yet (1.9) are the materialization of a very important concept: backpropagation.